Observación y Recolección de Información de los PROYECTOS DE FORMACIÓN DE LA FICHA 3214355

1. Observación de Proyectos

Es el proceso de analizar, no solo basta con revisar su apariencia o comprobar que sirve correctamente, es necesario analizar su código fuente, funcionamiento, estructura y calidad técnica.

Se debe verificar que el proyecto esté implementado y que cumpla con los objetivos establecidos.

2. Criterios para evaluar un Proyecto

Durante la observación de los proyectos se deben tener en cuenta los siguientes aspectos:

- Código Fuente: revisar que esté organizado, sea comprensible y funcione correctamente
- Arquitectura: Comprobar que el sistema tenga una estructura adecuada.
- Buenas prácticas: Verificar que se utilicen técnicas correctas de programación.
- Patrones de diseño: Identificar si se aplican soluciones de diseño apropiadas
- Pruebas Unitarias: Comprobar que existan pruebas para verificar el funcionamiento de los componentes.
- Estabilidad: Verificar que el sistema funcione sin errores constantes.
- Seguridad: revisar que los datos y accesos estén protegidos
- Integridad: Comprobar que la información se mantenga correcta y consistente.

Powered by CamScanner

- Herramientas de Evaluación (Rúbricas)
  - El proceso de evaluación se apoya en la generación automatizada de rúbricas
  - Los instrumentos de evaluación se generan en formatos estandarizados (PDF y HTML) y se organizan por fichas
  - Gestión Documental: El archivo Readme del repositorio de evaluación debe mantenerse limpio, limitando a enlazar la guía de evaluación y la presentación principal. Las rúbricas se regeneran y actualizan según sea necesario.
- Criterios Estrictos de Evaluación del Software (Fase de Ejecución)

La evaluación no es solo una lectura de código, es una revisión exhaustiva del sistema en funcionamiento

- Despliegue Obligatorio: El proyecto DEBE poder levantarse desde su código fuente. El evaluador debe poder ejecutar el proyecto, acceder a la aplicación y probar sus funcionalidades. No se evalúan capturas de pantalla.
- Idioma del Código: Exigencia del 100% en inglés (variables, funciones, clases, comentarios, etc.)
- Calidad y Arquitectura: Durante la revisión del código en funcionamiento se evalúa:
  - Implementación de Pruebas Unitarias
  - Coherencia de la Arquitectura del Software
  - Uso de Buenas prácticas de programación y Patrones de diseño
- Comportamiento del sistema: Se pondrá a prueba:
  - Estabilidad general del proyecto al interactuar con él
  - Integridad de los datos y del sistema
  - Seguridad del sistema (protección de la información y consistencia).
- Severidad: La revisión será estricta. Todo elemento mal elaborado, por pequeño que sea, será

Powered by CamScanner

documentado y sujeto a penalización

3. Criterios Estrictos de Evaluación de la Documentación

La documentación técnica tiene un peso evaluativo igual de riguroso que el código

- Formato y Presentación: Cumplimiento Con Normas Apa. se exige consistencia visual en todo el documento. Penalización: No justificado de negritas
- Recursos Visuales: Tablas e imágenes deben estar correctamente identificadas y citar sus fuentes
- Validación de Enlaces: Cero tolerancia a enlaces rotos, huérfanos o que no corresponden al contexto.
- Propiedad Intelectual e IA: Es obligatorio declarar el uso de IA Se penalizará encontrar metadatos de IA Se penalizará encontrar metadatos de IA no declarada o presentar contenido generado por IA como si fuera de autoría propia en ausencia de

4. Seguimiento de Contribuciones Individuales (Git)

La evaluación tiene un componente grupal (el producto final) y uno individual

- Informe Bibliométrico: Se exige un reporte detallado por la participación por integrante.

Métricas Requeridas

- Nombre y Usuarios de Git de Cada aprendiz
- Cronología detallada de Contribuciones
- Registro de los Commits/aportes realizados
- Mapa de Calor: Visualización del historial de actividad en el repositorio para auditar los momentos de trabajo de cada integrante.

Powered by CamScanner

Visibles por más

5. Flujo de Trabajo de la Evaluación Automatizada

El proceso de evaluación sigue un orden de ejecución (workflow) muy estructurado para revisar cada proyecto. Las etapas que pueden tomar varias horas por proyecto debido a la descarga de dependencias y levantamiento del sistema son:

1. Análisis estático y levantamiento del sistema: Revisión del código base y despliegue del proyecto.
2. Pruebas Unitarias y Cobertura: Ejecución de las pruebas automatizadas del sistema
3. Recorrido manual/funcional: Uso del sistema asumiendo roles para probar flujos específicos
4. Lectura de código fuente y Verificación adversarial: Revisión profunda mediante agentes para encontrar vulnerabilidades o fallos
5. Verificación de hallazgos y cálculo de notas: Consolidación de los errores encontrados
6. Generación de Documentos (PDF): Se emiten la Rúbrica diligenciada, el informe E2E y el informe de aportes consolidados

6. ENTORNO DE EJECUCIÓN Y PRIVACIDAD DE DATOS

- Contenedores: Los proyectos se levantan y evalúan utilizando entornos locales de Docker

- Limpieza de Datos: Se garantiza la privacidad eliminando los datos de prueba una vez finaliza la evaluación. Esto se hace destruyendo los volúmenes del contenedor mediante el comando.

7. Stack Tecnológico Oficial

Los proyectos formulados se enmarcan dentro de un stack tecnológico predefinido que abarca múltiples capas de arquitectura:

- Frontend / Experiencia de Usuario: Angular, Next.JS, Flutter.
- Backend & Compute: Java (Spring Boot), Go.
- Persistencia: MongoDB, Atlas, Redis, Couchbase.
- DevOps e Infraestructura: Github, Github Actions, Terraform.

8. Criterios de Pruebas Unitarias y Cobertura

La evaluación de las pruebas unitarias es rigurosa y se ejecuta directamente dentro de los contenedores. Se evalúan los siguientes metricas:

- Cobertura de Código: Se mide el porcentaje exacto de líneas y ramas de código que están cubiertas por las pruebas.
- Umbrales de aprobación: Cada módulo del sistema tiene un umbral mínimo exigido. Si no se cumple este umbral, el módulo no obtiene el veredicto de "Pass".

9. Verificación Funcional y Documentación de Fallos

- Se evalúan los flujos de usuario reales comprobando los códigos de respuesta.
- Si una funcionalidad no se logra probar o falla, queda explícitamente documentada en el informe MarkDown.

10. Entorno y Flujo de Evaluación

- Todo el proyecto se evalúa levantándolo en contenedores locales con docker compose.

Powered by CamScanner

- El evaluador automatizado sigue este orden:
Levantar sistema → Pruebas Unitarias → Recorrido Funcional (E2E) → Búsqueda de fallos → Generación de PDF.
- Privacidad: Al finalizar, el sistema borra todos los datos de prueba.

11. Stack tecnológico Sugerido
- Frontend: Angular, NextJS, Flutter
- Backend: Java, Go
- Base de datos: MongoDB, Redis
- Infraestructura: GitHub Actions y terraform, Azure, GCP

12. Reglas de Pruebas (Unitarias y E2E)
- Cobertura estricta: Se exige un porcentaje mínimo de código cubierto por pruebas; si el frontend o backend no llegan al umbral, se penaliza la nota.
- Cero pruebas falsas: Hacer pruebas que no llamen al código real del producto se detecta y se penaliza.
- Pruebas funcionales (E2E): Se simulan usuarios reales interactuando con la aplicación levantada. Si un flujo falla, los procesos que dependían de ese login no se evalúan y quedan omitidos.
- Regla de Oro: El evaluador Nunca modifica el código, ni arregla el archivo .env, ni reinicia servicios para que tu proyecto funcione.

13. Penalización de Errores

P0: Fallo grave, el sistema no arranca. Reprueba el proyecto.

P1: El criterio queda calificado como "No cumple".

P2 y P3: El criterio baja a "Parcial".

Powered by CamScanner

14. Requisitos para Aprobar

1. Tener Pull Request y Release
2. Repositorio Accesible
3. El sistema debe desplegarse a la app y levantar
4. Cero contraseñas o secretos expuestos en el código
5. Sacar al menos la mitad de los puntos en ejecución y seguridad
6. Tener Cero errores críticos abiertos.